Alumno: Emiliano Martinez Torres

BITÁCORA DE IA

PARTE 1

Prompt utilizado: ”Analiza la captura proporcionada y extrae los atributos, métodos y relaciones de las clases Vehiculo, CamionCarga, AutobusPasajeros y Flota. Identifica cuál debe ser abstracta y qué método debe ser polimórfico.”

Lo que entregó la ia:

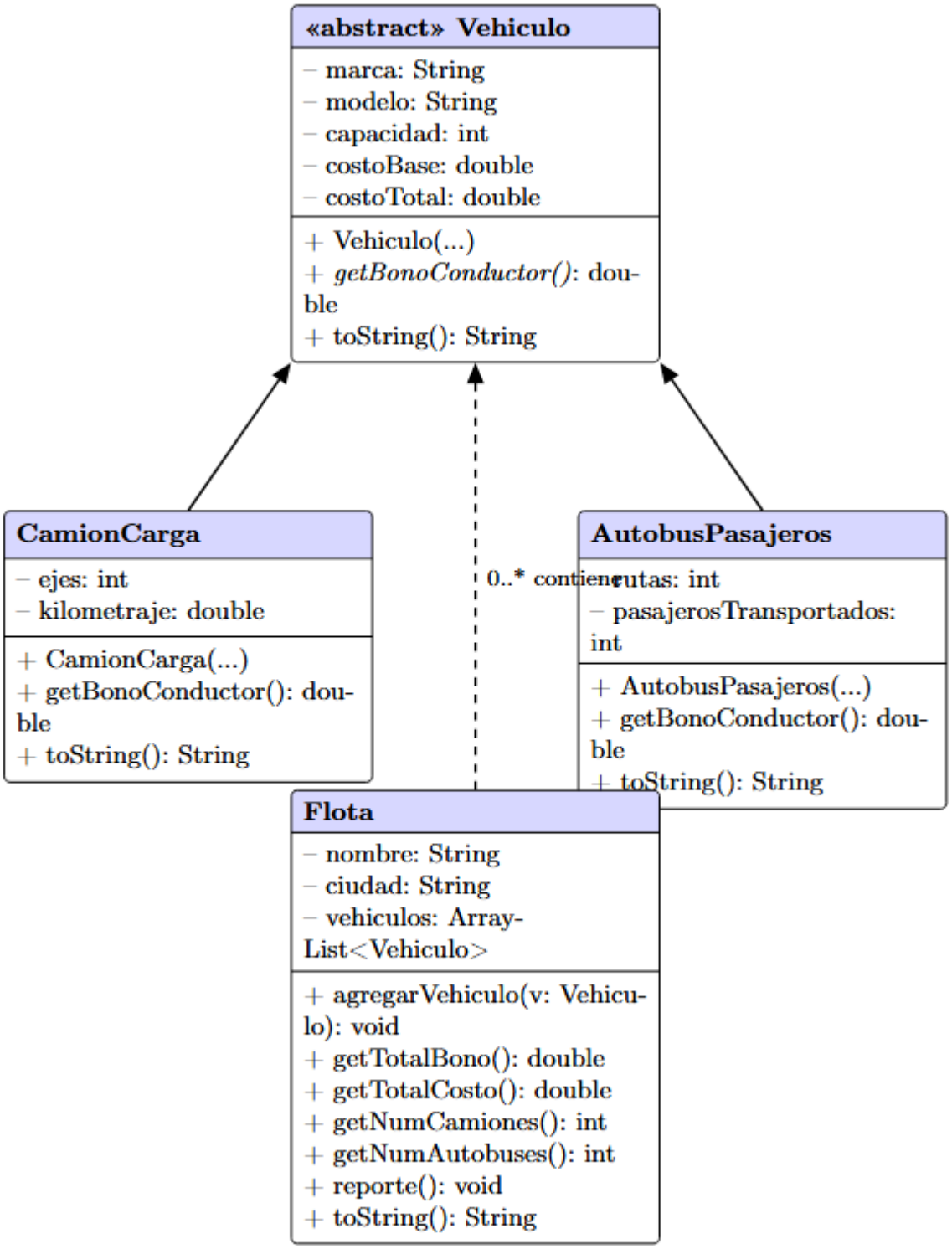
identificación de la jerarquía, atributos privados, método abstracto getBonoConductor()
y el uso de ArrayList<Vehiculo> en Flota. (Asi tal cual me lo entregó la ia me empece a preocupar porque pensé que no me entendía pero conforme le di los demás prompts se le fue dando más forma seguimos con la explicación de lo realizado)

PARTE 2

Prompt utilizado: “Genera el diagrama UML de clases para el sistema de flota, mostrando herencia, clase abstracta, atributos privados, métodos públicos y la relación de composición entre Flota y Vehiculo.”

Lo que entregó la ia:

Alumno: Emiliano Martinez Torres



Fórmulas de bono (polimorfismo)
CamionCarga: Bono = (CostoBase × 0,10) + (Ejes × 50) + (Kilometraje × 0,5)
AutobusPasajeros: Bono = (CostoBase × 0,15) + (Rutas × 120) + (P asajeros × 0,1)

Para este punto vi que ya íbamos llevando el proyecto por buen camino asi que segui con las siguientes instrucciones de las siguiente manera.

Alumno: Emiliano Martinez Torres

PARTE 3

Prompt utilizado: “Implementa en Java la clase abstracta Vehiculo con atributos privados, constructor que inicialice marca, modelo, capacidad y costoBase, y el método abstracto getBonoConductor(). Incluye toString().”

Lo que entregó la ia:

```java
/**
 * Clase abstracta base que representa un vehículo genérico de la flota.
 * Define los atributos y comportamientos comunes a todos los vehículos.
 */
public abstract class Vehiculo {

    // ---------- Atributos comunes ----------
    private String marca;
    private String modelo;
    private int capacidad;       // kg o pasajeros según el tipo
    private double costoBase;    // costo base de operación mensual
    private double costoTotal;   // se calcula en el constructor derivado

    // ---------- Constructor ----------
    public Vehiculo(String marca, String modelo, int capacidad, double costoBase) {
        this.marca = marca;
        this.modelo = modelo;
        this.capacidad = capacidad;
        this.costoBase = costoBase;
        this.costoTotal = 0.0; // se sobrescribe en la clase derivada
    }

    // ---------- Método abstracto (polimorfismo) ----------
    /**
     * Calcula el bono específico del conductor según el tipo de vehículo.
     * Cada subclase DEBE implementarlo con su propia fórmula.
     */
    public abstract double getBonoConductor();

    // ---------- Getters / Setters ----------
```

Alumno: Emiliano Martinez Torres

```
    public String getMarca() { return marca; }
    public void setMarca(String marca) { this.marca = marca; }

    public String getModelo() { return modelo; }
    public void setModelo(String modelo) { this.modelo = modelo; }

    public int getCapacidad() { return capacidad; }
    public void setCapacidad(int capacidad) { this.capacidad =
capacidad; }

    public double getCostoBase() { return costoBase; }
    public void setCostoBase(double costoBase) { this.costoBase =
costoBase; }

    public double getCostoTotal() { return costoTotal; }
    protected void setCostoTotal(double costoTotal) { this.costoTotal =
costoTotal; }

    // ---------- toString sobrecargado ----------
    @Override
    public String toString() {
        return String.format(
            "Vehículo [Marca=%s, Modelo=%s, Capacidad=%d,
CostoBase=%.2f]",
            marca, modelo, capacidad, costoBase
        );
    }
}
```

PARTE 4

Prompt utilizado: “Implementa CamionCarga y AutobusPasajeros extendiendo Vehiculo. Aplica las fórmulas de bono dadas, calcula CostoTotal en el constructor (CostoBase + Bono) y sobrescribe toString() con @Override.”

Lo que entregó la ia:

```
/**
 * Camión de carga. Hereda de Vehiculo e implementa su propio bono.
 * Bono = (CostoBase * 0.10) + (Ejes * 50) + (Kilometraje * 0.5)
 */
public class CamionCarga extends Vehiculo {
```

Alumno: Emiliano Martinez Torres

```
    private int ejes;
    private double kilometraje;

    public CamionCarga(String marca, String modelo, int capacidad,
                       double costoBase, int ejes, double kilometraje)
{
        super(marca, modelo, capacidad, costoBase);
        this.ejes = ejes;
        this.kilometraje = kilometraje;
        // Cálculo del costo total usando el bono polimórfico
        setCostoTotal(getCostoBase() + getBonoConductor());
    }

    @Override
    public double getBonoConductor() {
        return (getCostoBase() * 0.10) + (ejes * 50) + (kilometraje *
0.5);
    }

    // ---------- Getters / Setters ----------
    public int getEjes() { return ejes; }
    public void setEjes(int ejes) { this.ejes = ejes; }

    public double getKilometraje() { return kilometraje; }
    public void setKilometraje(double kilometraje) { this.kilometraje =
kilometraje; }

    @Override
    public String toString() {
        return String.format(
            "CamionCarga [%s, Ejes=%d, Kilometraje=%.1f, Bono=%.1f,
CostoTotal=%.1f]",
            super.toString(), ejes, kilometraje, getBonoConductor(),
getCostoTotal()
        );
    }
}
```

También me entrego lo siguiente con este mismo prompt:

```
/**
```

Alumno: Emiliano Martinez Torres

```
 * Autobús de pasajeros. Hereda de Vehiculo e implementa su propio
bono.
 * Bono = (CostoBase * 0.15) + (Rutas * 120) + (PasajerosTransportados
* 0.1)
 */
public class AutobusPasajeros extends Vehiculo {

    private int rutas;
    private int pasajerosTransportados;

    public AutobusPasajeros(String marca, String modelo, int capacidad,
                            double costoBase, int rutas, int
pasajerosTransportados) {
        super(marca, modelo, capacidad, costoBase);
        this.rutas = rutas;
        this.pasajerosTransportados = pasajerosTransportados;
        setCostoTotal(getCostoBase() + getBonoConductor());
    }

    @Override
    public double getBonoConductor() {
        return (getCostoBase() * 0.15) + (rutas * 120) +
(pasajerosTransportados * 0.1);
    }

    // ---------- Getters / Setters ----------
    public int getRutas() { return rutas; }
    public void setRutas(int rutas) { this.rutas = rutas; }

    public int getPasajerosTransportados() { return
pasajerosTransportados; }
    public void setPasajerosTransportados(int pasajerosTransportados) {
        this.pasajerosTransportados = pasajerosTransportados;
    }

    @Override
    public String toString() {
        return String.format(
            "AutobusPasajeros [%s, Rutas=%d, PasajerosTransportados=%d,
Bono=%.1f, CostoTotal=%.1f]",
            super.toString(), rutas, pasajerosTransportados,
            getBonoConductor(), getCostoTotal()
        );
```

Alumno: Emiliano Martinez Torres

```
    }
}
```

PARTE 5

Prompt utilizado: “Implementa la clase Flota con ArrayList<Vehiculo>, métodos agregarVehiculo, getTotalBono, getTotalCosto, getNumCamiones, getNumAutobuses y reporte. En App.java agrega un comentario explicando el late binding.”

Lo que entregó la ia:

```
import java.util.ArrayList;

/**
 * Gestiona la flota de vehículos. Usa un ArrayList<Vehiculo>
 * para almacenar objetos de las subclases (polimorfismo).
 */
public class Flota {

    private String nombre;
    private String ciudad;
    private ArrayList<Vehiculo> vehiculos;

    public Flota(String nombre, String ciudad) {
        this.nombre = nombre;
        this.ciudad = ciudad;
        this.vehiculos = new ArrayList<>();
    }

    public void agregarVehiculo(Vehiculo vehiculo) {
        vehiculos.add(vehiculo);
    }

    /** Suma de todos los bonos (llamada polimórfica a
getBonoConductor). */
    public double getTotalBono() {
        double total = 0.0;
        for (Vehiculo v : vehiculos) {
            total += v.getBonoConductor();
        }
        return total;
    }
```

Alumno: Emiliano Martinez Torres

```
    /** Suma de todos los costos totales. */
    public double getTotalCosto() {
        double total = 0.0;
        for (Vehiculo v : vehiculos) {
            total += v.getCostoTotal();
        }
        return total;
    }

    /** Cuenta cuántos vehículos son instancias de CamionCarga. */
    public int getNumCamiones() {
        int cont = 0;
        for (Vehiculo v : vehiculos) {
            if (v instanceof CamionCarga) cont++;
        }
        return cont;
    }

    /** Cuenta cuántos vehículos son instancias de AutobusPasajeros. */
    public int getNumAutobuses() {
        int cont = 0;
        for (Vehiculo v : vehiculos) {
            if (v instanceof AutobusPasajeros) cont++;
        }
        return cont;
    }

    /** Imprime el reporte detallado de cada vehículo. */
    public void reporte() {
        System.out.println(">> Reporte Detallado de la Flota <<");
        for (Vehiculo v : vehiculos) {
            System.out.println(v.toString());
        }
        System.out.println();
    }

    @Override
    public String toString() {
        return String.format(
            "Flota [Nombre=%s, Ciudad=%s, Vehículos=%d, BonoTotal=%.1f,
CostoTotal=%.1f, Camiones=%d, Autobuses=%d]",
            nombre, ciudad, vehiculos.size(),
```

Alumno: Emiliano Martinez Torres

```
            getTotalBono(), getTotalCosto(),
            getNumCamiones(), getNumAutobuses()
        );
    }
}
```

También me entrego lo siguiente con este mismo prompt:

```
/**
 * Clase principal del sistema de gestión de flota.
 *
 * ============================================================
 *  EXPLICACIÓN TÉCNICA: LATE BINDING (ENLACE DINÁMICO)
 * ============================================================
 * En este programa se utiliza LATE BINDING (o dynamic binding)
 * gracias al uso de polimorfismo con la clase abstracta Vehiculo.
 *
 * Cuando declaramos:
 *      Vehiculo v = new CamionCarga(...);
 *      Vehiculo v2 = new AutobusPasajeros(...);
 *
 * el tipo de referencia es Vehiculo, pero el objeto real en
 * memoria es de la subclase correspondiente. Al invocar:
 *
 *      v.getBonoConductor();
 *      v.getCostoTotal();
 *
 * la JVM no resuelve el método en tiempo de compilación, sino
 * en TIEMPO DE EJECUCIÓN (late binding), seleccionando la
 * implementación correcta según el tipo real del objeto.
 *
 * Esto permite que en Flota.reporte() o Flota.getTotalBono()
 * se recorran todos los vehículos del ArrayList<Vehiculo> y se
 * ejecute automáticamente la versión del método que corresponde
 * a cada subclase (CamionCarga o AutobusPasajeros), sin usar
 * condicionales instanceof para calcular el bono.
 * ============================================================
 */
public class App {

    public static void main(String[] args) {

        // -------- Reporte Inicial (flota vacía) --------
```

Alumno: Emiliano Martinez Torres

```
        System.out.println("--- Reporte Inicial ---");
        Flota flota = new Flota("División Norte", "Monterrey");
        System.out.println(flota.toString());
        System.out.println();

        // -------- Reporte Final con datos de prueba --------
        System.out.println("--- Reporte Final con Datos de Prueba
---");

        flota.agregarVehiculo(new CamionCarga("Volvo", "FMX", 30000,
2500.00, 4, 5000.0));
        flota.agregarVehiculo(new CamionCarga("Kenworth", "T680",
25000, 1800.00, 3, 4200.0));

        flota.agregarVehiculo(new AutobusPasajeros("Mercedes", "0500",
50, 1200.00, 5, 8500));
        flota.agregarVehiculo(new AutobusPasajeros("Irizar", "i6", 40,
1500.00, 8, 12000));
        flota.agregarVehiculo(new AutobusPasajeros("Scania", "K360",
45, 950.00, 4, 6200));

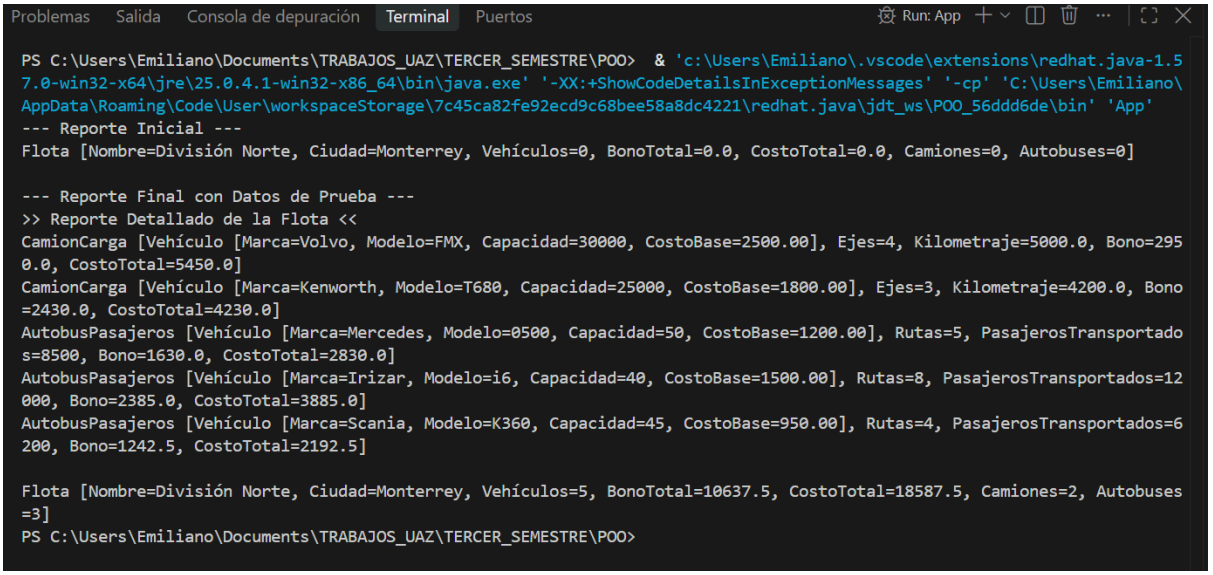
        flota.reporte();

        System.out.println(flota.toString());
    }
}
```

Alumno: Emiliano Martinez Torres

RESULTADO

A continuación pongo una captura de pantalla del resultado que me dio al ejecutar el codigo obtenido a lo largo de los prompts utilizados:



CONCLUSIÓN

Como conclusión obtengo que la ayuda de la ia fue buena pero aunque no lo coloque también tuvo algunos errores solo que fueron cosas simples de detectar y corregir por eso no los agregue, también obtengo que el uso de la clase abstracta así como del polimorfismo son bastante útiles a la hora de realizar este tipo de problemas como conclusión general fue bastante bueno el desarrollo del proyecto ya que sirvió para poder poner en práctica en primer lugar el uso del polimorfismo y de las clases abstractas y en segundo la capacidad de poder leer código ajeno (En este caso el código proporcionado por la ia) en busca de errores o formas de mejorar el código (Lo cual se logró ya que logre detectar los errores y mejorar en algunas partes el código).